



Παράλληλος και Κατανεμημένος Υπολογισμός I

Μοιραζόμενη μνήμη
Αμοιβαίος αποκλεισμός

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025

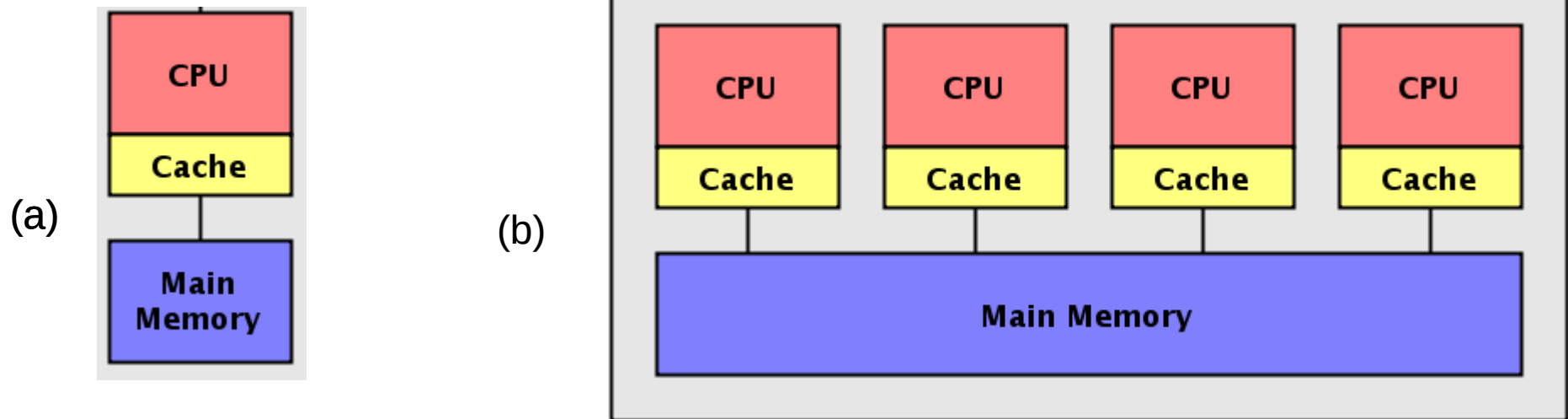


Μοιραζόμενη μνήμη

Μοιραζόμενη μνήμη

- Οι χώροι διευθύνσεων των διεργασιών ή νημάτων έχουν μοιραζόμενες περιοχές, δηλαδή περιοχές που αναφέρονται στις ίδιες θέσεις φυσικής μνήμης.
- Ο ταυτόχρονος υπολογισμός παρέχει μηχανισμούς που επιτρέπουν την προστασία και τον συντονισμό στη πρόσβαση των μοιραζόμενων θέσεων μνήμης.
- Οι ίδιοι μηχανισμοί μπορούν να χρησιμοποιηθούν και στον παράλληλο υπολογισμό μοιραζόμενης μνήμης.
- Η χρήση των μηχανισμών κοστίζει σε χρόνο και πιθανά σφάλματα. Στόχος μας είναι η απολύτως απαραίτητη και προσεκτική χρήση των μηχανισμών.

Μοντέλο υπολογιστικού συστήματος μοιραζόμενης μνήμης



(a) Μοντέλο υπολογιστικού συστήματος ενός επεξεργαστή με κρυφή και κύρια μνήμη.

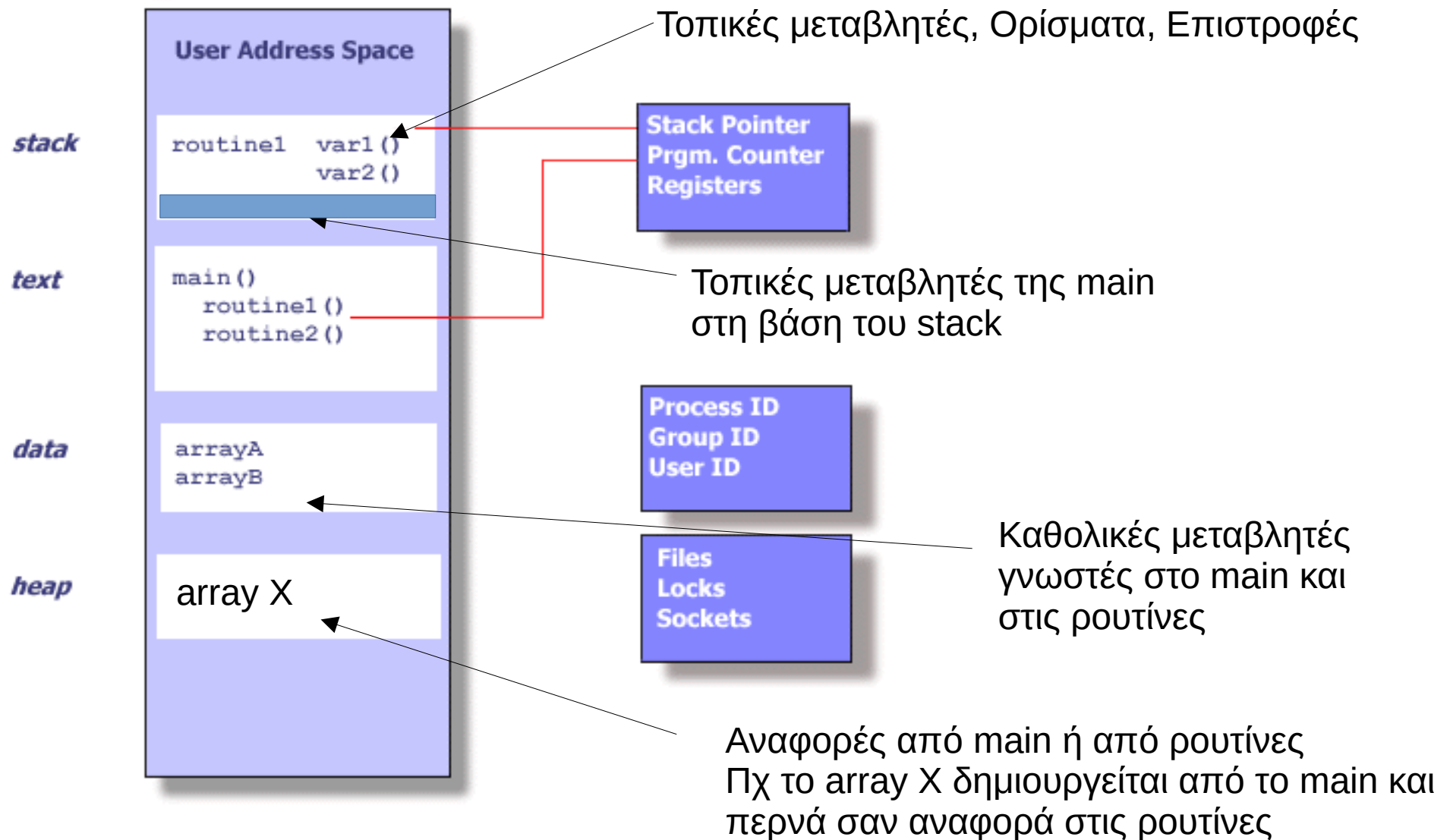
(b) Μοντέλο υπολογιστικού συστήματος κοινόχρηστης ή μοιραζόμενης μνήμης.

Στην κύρια μνήμη μπορεί να βρίσκονται πολλές διεργασίες, μερικές με κοινόχρηστες μεταβλητές. Επίσης μπορεί να βρίσκονται διεργασίες που η κάθε μια να έχει πολλά νήματα (τα οποία έχουν κοινό τον χώρο διευθύνσεων της διεργασίας).

Μοιραζόμενες μεταβλητές

- Μια μεταβλητή που έχει εμβέλεια η οποία καλύπτει δύο ή περισσότερες διεργασίες / νήματα είναι μοιραζόμενη (κοινόχρηστη).
- Ισχύουν οι κανόνες εμβέλειας / διάρκειας ζωής μεταβλητών και περάσματος ορισμάτων / παραμέτρων του ακολουθιακού προγραμματισμού.
- Αναφερόμαστε κυρίως σε νήματα: οι μοιραζόμενες μεταβλητές διεργασιών είναι κάπως πιο περίπλοκες στον χειρισμό τους, αλλά η λογική είναι ίδια.

Χώρος διευθύνσεων: διεργασία



Κάθε στιγμή εκτελείται ΜΟΝΟ μια ρουτίνα ή το `main()`. ΟΧΙ όλα ταυτόχρονα.

Στοίβα κλήσεων (στοίβα εκτέλεσης)

```
int[] arrayA[10];  
int[] arrayB[10];
```

```
routine1 (int y, int[] x) {  
    int var1;  
    int var2;  
    ...  
}
```

```
routine2 (int z, int[] Y) {  
    int var1;  
    int var2;  
    int var3;  
    ...  
}
```

```
main() {  
    int a;  
    int[] arrayX[10];  
    ...  
    routine1(a, arrayX);  
    routine2(a, arrayX);  
    ...  
}
```

Στοίβα εκτέλεσης: χρονική εξέλιξη

int a; int[] arrayX;

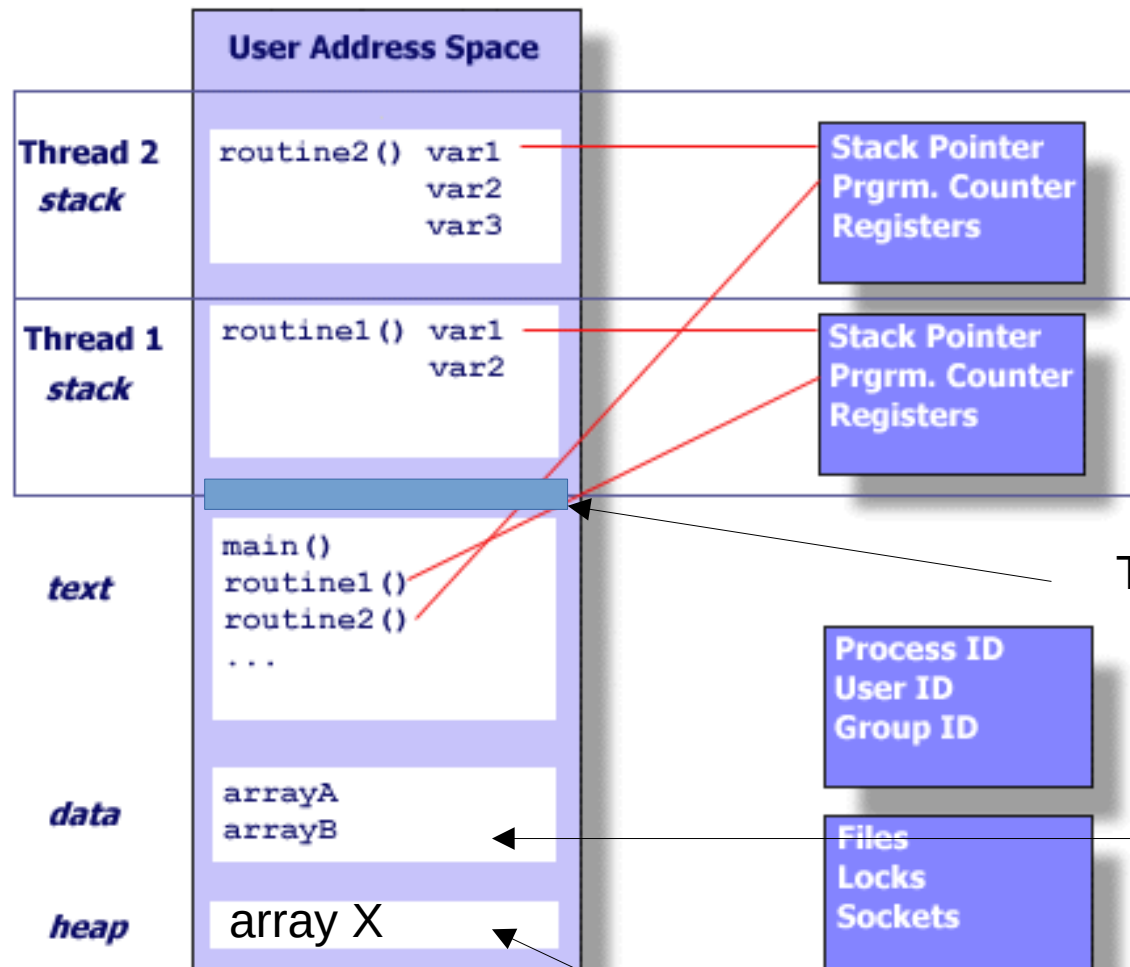
int y = value of a;
int[] X = reference to arrayX
int var1; int var2;

int z = value of a;
int[] Y = reference to arrayX
int var1; int var2; int var3;

int a; int[] arrayX;

Οι καθολικές μεταβλητές συνεχώς ορατές

Χώρος διευθύνσεων: διεργασία με νήματα



Αν το `main` περάσει το `X` σαν όρισμα αναφοράς στο `Thread1` τότε το `main()` και το `Thread1` μπορούν να τροποποιήσουν το `X`

Αν το `main` περάσει το `X` σαν όρισμα αναφοράς και στα δύο νήματα τότε όλοι μπορεί να τροποποιήσουν το `X`

Το `main` έχει δικό του `stack`

Οι καθολικές μεταβλητές είναι ορατές από το `main` και τα νήματα. Όλοι μπορεί να τις τροποποιήσουν.

Το `array X` δημιουργείται από το `main` και περνά σαν αναφορά στα νήματα

Κάθε στιγμή μπορεί να εκτελούνται ΤΑΥΤΟΧΡΟΝΑ το `main()` και δύο νήματα.

Συνηθισμένοι τρόποι διαμοιρασμού μεταβλητών

- **Ορίσματα** που περνούμε στα νήματα κατά τη δημιουργία τους, συνήθως μέσω του κύριου προγράμματος, που είναι ο δημιουργός των νημάτων. Η συνηθέστερη μέθοδος διαμοιρασμού.
- **Καθολικές μεταβλητές**, γνωστές εξ' ορισμού σε όλο το πρόγραμμα. Εύκολη λύση αλλά η κάθε γλώσσα χειρίζεται λίγο διαφορετικά τις καθολικές μεταβλητές. Προσοχή στο διαφορετικό συντακτικό των καθολικών μεταβλητών στις διάφορες γλώσσες.

Μοιραζόμενες μεταβλητές

Ορίσματα τιμής και ορίσματα αναφοράς

- **Ορίσματα τιμής:** πέρασμα τιμής από τον καλούντα (συνήθως το `main()`) στον καλούμενο (στο νήμα που δημιουργείται). **Όχι** όμως ταυτόχρονη τροποποίηση. Από την στιγμή που περάσει η τιμή στο νήμα, αυτή αποθηκεύεται σε τοπική μεταβλητή του νήματος.
- **Ορίσματα αναφοράς:** πέρασμα αναφοράς σε μεταβλητή / δομή / αντικείμενο από τον καλούντα στον καλούμενο. **Δυνατότητα** για ταυτόχρονη τροποποίηση. Καλών, καλούμενος (ή και πολλοί καλούμενοι) “βλέπουν” τις ίδιες (μοιραζόμενες) θέσεις μνήμης. Απαιτείται προσοχή και προστασία.

Μοιραζόμενες μεταβλητές

Απλές μεταβλητές, δομές και αντικείμενα

- **Οι δομές δεδομένων** (πχ πίνακες) και τα αντικείμενα (objects, structures..) συνήθως ορίζονται ως αναφορές (δείκτες σε..). Επομένως όταν περνούν ως ορίσματα είναι εξ' ορισμού ορίσματα αναφοράς.
- **Οι απλές μεταβλητές** (πχ int) μπορεί να ορίζονται ως απλές τιμές (όχι δείκτες σε..). Επομένως όταν περνούν ως ορίσματα μπορεί να είναι ορίσματα τιμής. Μπορούν να μετατραπούν σε ορίσματα αναφοράς με χρήση δεικτών ή μέσω της ενσωμάτωσης σε κάποια δομή ή αντικείμενο.

Μοιραζόμενες μεταβλητές: σύνοψη

	Απλές μεταβλητές	Δομές και αντικείμενα
Ορίσματα	Τιμή ή Αναφορά	Αναφορά
Καθολικές μεταβλητές	Εμβέλεια	Εμβέλεια

Μοιραζόμενες μεταβλητές: παράδειγμα

```
int[] arrayA[10];  
int[] arrayB[10];
```

Καθολικές μεταβλητές ορατές σε
όλα τα νήματα

```
routine1 (int y, int[] x) {  
    int var1;  
    int var2;  
    ...  
}
```

```
int a; int[] arrayX;
```

```
routine2 (int z, int[] Y) {  
    int var1;  
    int var2;  
    int var3;  
    ...  
}
```

```
int y = value of a;  
int[] X = reference to arrayX  
int var1; int var2;
```

```
main() {  
    int a;  
    int[] arrayX[10];  
    ...  
    create/start thread(routine1(a, arrayX));  
    create/start thread(routine2(a, arrayX));  
    ...  
}
```

```
int z = value of a;  
int[] Y = reference to arrayX  
int var1; int var2; int var3;
```

Στοίβες εκτέλεσης: ανεξάρτητες μετά τη
δημιουργία τους

Καθολικές μεταβλητές

```
int my_data = 0;  // Global variable C
```

```
ThreadA () {  
    my_data = 1;  
    ...  
}
```

```
ThreadB () {  
    int local_var;  
    local_var = my_data;  
    ...  
}
```

```
main() {  
    ...  
    create/start thread(ThreadA());  
    create/start thread(ThreadB());  
    ...  
}
```

Όρισμα αναφοράς

```
ThreadA (int *x) {  
    *x = 1;  
    ...  
}
```

```
ThreadB (int *x) {  
    int local_var;  
    local_var = *x;  
    ...  
}
```

```
main() {  
    my_data = 0;  
    create/start thread(ThreadA(&my_data)); //shared reference  
    create/start thread(ThreadB(&my_data));  
    ...  
}
```

Εδώ τα σύμβολα *,& έχουν την σημασία της C αλλά μπορεί να είναι διαθέσιμα σε άλλες γλώσσες με διαφορετικούς τρόπους.

Αντικείμενο/δομή ως όρισμα αναφοράς

```
struct/class shared_data {  
    int my_data = 0;  
}
```

```
ThreadA (shared_data x) {  
    x.my_data = 1;  
    ...  
}
```

```
ThreadB (shared_data x) {  
    int local_var;  
    local_var = x.my_data;  
    ...  
}
```

```
main() {  
    shared_data obj = new shared_data();  
    create/start thread(ThreadA(obj)); // shared reference  
    create/start thread(ThreadB(obj));  
    ...  
}
```

Μοιραζόμενες μεταβλητές

Στοίχιση τιμών

Συνήθεις λειτουργίες ορισμάτων τιμής:

- Πέρασμα σε όλα τα νήματα κοινών τιμών, πχ το πλήθος των νημάτων.
- Πέρασμα σε κάθε νήμα ενός ID, πχ νήμα 0, νήμα 1, νήμα 2 κλπ.
- Η δεύτερη τεχνική λέγεται στοίχιση, δηλαδή το όρισμα τιμής κάθε νήματος στοιχίζεται σε μια διαφορετική τιμή που δίνει το `main`.

Μοιραζόμενες μεταβλητές

Επιστροφές τιμών

- Όταν δεν υπάρχουν ορίσματα αναφοράς πρέπει να γίνει επιστροφή τιμών από τον καλούμενο στον καλούντα.
- Σε μερικές βιβλιοθήκες (πχ Posix Threads C/C++) δεν υπάρχει ρητή επιστροφή τιμών.
- Στη Java, στα απλά νήματα, μπορεί να γίνει μέσω μεταβλητών αντικειμένου που είναι γνωστές στον καλούντα (συνήθως το main). Καλό είναι οι επιστροφές να γίνονται μέσω get().



Μοιραζόμενες μεταβλητές

Ταυτόχρονη πρόσβαση μέσω ορισμάτων αναφοράς

- Απαιτείται προσοχή στη χρήση ή και προστασία.
- Το ένα νήμα μπορεί να επηρεάσει τη δουλειά του άλλου (συμπεριλαμβανομένου και του main)
- Μια λύση είναι κάθε νήμα να γράφει σε ξεχωριστή περιοχή της δομής δεδομένων. Η μοιραζόμενη ανάγνωση δεν μας προβληματίζει, αλλά όταν υπάρχει ανάγνωση-εγγραφή ή εγγραφή-εγγραφή έχουμε πρόβλημα.
- Θα το δούμε αναλυτικά παρακάτω..

Κανόνες χρήσης μοιραζόμενων μεταβλητών

- Περιορίζουμε στο απολύτως απαραίτητο τη χρήση των μοιραζόμενων μεταβλητών: Προσπαθούμε να έχουμε όσο το δυνατό περισσότερες (α) τοπικές μεταβλητές συναρτήσεων/μεθόδων και (β) τοπικές μεταβλητές νημάτων.
- Τεκμηριώνουμε καλά τη χρήση μοιραζόμενων μεταβλητών που μπορεί να έχουν παρενέργειες, δηλαδή καθολικές μεταβλητές, παράμετροι αναφοράς στο νήμα.
- Προσπαθούμε να έχουμε επικοινωνία με μοιραζόμενες μεταβλητές μόνο σε περιορισμένα σημεία του κώδικα, ιδανικά στην έναρξη και στον τερματισμό των νημάτων.



Συνθήκες συναγωνισμού και κρίσιμα τμήματα

Συνθήκη συναγωνισμού (Race condition)

- Το φαινόμενο της αδυναμίας καθορισμού της σειράς εκτέλεσης εντολών που εκτελούνται ταυτόχρονα και τροποποιούν μοιραζόμενο πόρο ώστε μπορούν να μας οδηγήσουν σε “εσφαλμένα” αποτελέσματα.
- Παράδειγμα: πολλά νήματα προσπαθούν εκτελέσουν ακολουθίες εκτυπώσεων (μοιραζόμενος πόρος: η οθόνη-- μέσω των κλήσεων συστήματος).
- Οι εκτυπώσεις κάθε νήματος είναι ακολουθιακά συνεπείς αλλά η συνολική σειρά των εκτυπώσεων δεν είναι, δηλαδή είναι απρόβλεπτη.

Παράδειγμα συνθήκης συναγωνισμού

- Έστω δυο νήματα A και B που εκτελούν ταυτόχρονα τον παρακάτω κώδικα

main:

```
create/start ThreadA  
create/start ThreadB
```

ThreadA :

```
...  
print("Hello from ThreadA")  
print("Bye from ThreadA")  
...
```

ThreadB :

```
...  
print("Hello from ThreadB")  
print("Bye from ThreadB")  
...
```

Ποιες είναι οι πιθανές σειρές εκτυπώσεων;
Ποιες θεωρούμε “σωστές”;

Κρίσιμο τμήμα (Critical section)

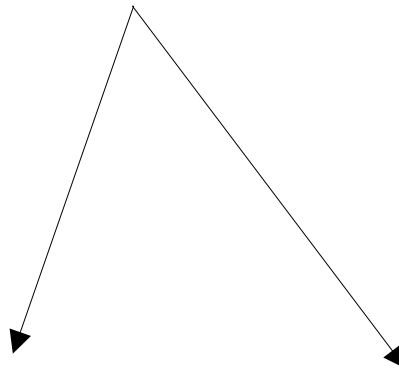
- Το τμήμα του κώδικα που είναι υπεύθυνο για τη συνθήκη συναγωνισμού.
- Στο παράδειγμα μας: οι γραμμές κώδικα που περιλαμβάνουν την ακολουθία εκτυπώσεων κάθε νήματος.

ThreadA :

```
...  
print("Hello from ThreadA")  
print("Bye from ThreadA")  
...
```

ThreadB :

```
...  
print("Hello from ThreadB")  
print("Bye from ThreadB")  
...
```



Συναγωνισμός δεδομένων (Data race)

- Το φαινόμενο της αδυναμίας καθορισμού της σειράς εκτέλεσης εντολών που εκτελούνται ταυτόχρονα και τροποποιούν **μοιραζόμενες μεταβλητές** ώστε μπορούν να μας οδηγήσουν σε “εσφαλμένα” αποτελέσματα.
- Το τμήμα του κώδικα που είναι υπεύθυνο για το συναγωνισμό δεδομένων λέγεται κρίσιμο τμήμα.

Πώς αναγνωρίζουμε το συναγωνισμό δεδομένων και τα αντίστοιχα κρίσιμα τμήματα; 1

- Ελέγχουμε τις μοιραζόμενες μεταβλητές για κάθε νήμα.
- Στον κώδικα κάθε νήματος βρίσκουμε τις εντολές που περιέχουν μοιραζόμενες μεταβλητές.
- Αν μια μοιραζόμενη μεταβλητή σε όλους τους κώδικες **μόνο** διαβάζεται ταυτόχρονα

ανάγνωση ? ανάγνωση

τότε για τη συγκεκριμένη μεταβλητή δεν υφίσταται συνθήκη συναγωνισμού.

Πώς αναγνωρίζουμε το συναγωνισμό δεδομένων και τα αντίστοιχα κρίσιμα τμήματα; 2

- Αν μια μοιραζόμενη μεταβλητή διαβάζεται και γράφεται ταυτόχρονα *

ανάγνωση ? εγγραφή

ή γράφεται ταυτόχρονα

εγγραφή ? εγγραφή

- τότε για τη συγκεκριμένη μεταβλητή υφίσταται συναγωνισμός δεδομένων.
- Οι εντολές που προκαλούν συναγωνισμό δεδομένων είναι το κρίσιμο τμήμα.

*Προσοχή: εδώ αναφερόμαστε σε **μια μεταβλητή** όχι πχ σε πίνακα που κάθε νήμα πιθανώς γράφει σε άλλη θέση

Σύνοψη συναγωνισμού δεδομένων

Ανάγνωση	Ανάγνωση	ΟΚ
Ανάγνωση	Εγγραφή	Συναγωνισμός Δεδομένων
Εγγραφή	Εγγραφή	Συναγωνισμός Δεδομένων

Παράδειγμα συναγωνισμού δεδομένων 1

```
//Shared variable  
my_data = 0
```

Main:

```
create/start ThreadA  
create/start ThreadB
```

ThreadA :

```
...
```

```
my_data = 1
```

```
...
```

ThreadB :

```
local_var = 0
```

```
...
```

```
if my_data == 1...  
(ή local_var = my_data)  
(ή my_data = 2)
```

```
...
```

Παράδειγμα συναγωνισμού δεδομένων 2

Εδώ τα πιθανά σφάλματα σχετίζονται με το γεγονός ότι τόσο ο μεταγλωττιστής όσο και το σύστημα χρόνου εκτέλεσης δεν μπορούν να έχουν γνώση της επιθυμητής σειράς εκτέλεσης από την πλευρά του προγραμματιστή.

Για παράδειγμα:

Ποια ανάθεση πρέπει να συμβεί πρώτη;
Ποια τιμή πρέπει να λάβει η τοπική μεταβλητή;
Πότε πρέπει να εκτελεστεί το if;

Ο προγραμματιστής πρέπει να καθορίσει τη σειρά εκτέλεσης. Θα το δούμε στο συγχρονισμό σε συνθήκες.

Σημείωση: τι σημαίνει Shared variable?

Στους σκελετούς κώδικα που ακολουθούν, για λόγους απλότητας χρησιμοποιούμε το

```
//Shared variable
```

Όμως πρέπει να θυμόμαστε ότι στην πραγματικότητα απαιτείται εφαρμογή συγκεκριμένης τεχνικής διαμοιρασμού.

Καλή προγραμματιστική πρακτική:
Συνεπής χρήση όσο το δυνατόν μιας τεχνικής διαμοιρασμού.

Παρεμβολή: συναγωνισμός δεδομένων υψηλού επιπέδου 1

- Αν μια μοιραζόμενη μεταβλητή ενημερώνεται ταυτόχρονα

ανάγνωση(-τροποποίηση)-εγγραφή ?
ανάγνωση(-τροποποίηση)-εγγραφή

τότε για τη συγκεκριμένη μεταβλητή υφίσταται συνθήκη συναγωνισμού που λέγεται **παρεμβολή ή συναγωνισμός δεδομένων υψηλού επιπέδου.**

- Οι εντολές που προκαλούν παρεμβολή είναι το κρίσιμο τμήμα.

Παράδειγμα παρεμβολής

- Έστω δυο νήματα A και B που εκτελούν ταυτόχρονα τον παρακάτω κώδικα

```
// shared variable  
counter = 0
```

```
main:  
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :  
    ...  
    counter = counter + 1  
    ...
```

```
ThreadB :  
    ...  
    counter = counter + 1  
    ...
```

Παρεμβολή: ανάλυση σε ατομικές λειτουργίες 1

- Μια απλή πρόταση γλώσσας υψηλού επιπέδου, για παράδειγμα:

counter = counter + 1

μεταγλωττίζεται σε ακολουθία εντολών γλώσσας μηχανής.

- Οι εντολές γλώσσας μηχανής είναι κατά κανόνα **ατομικές (αδιαίρετες) λειτουργίες**, δηλαδή λειτουργίες που δε μπορεί να διακοπούν- είτε εκτελούνται συνολικά ή καθόλου:

load [mem], r ; load memory location to register

inc r ; increment register

store [mem], r ; store register to memory location

Παρεμβολή: ανάλυση σε ατομικές λειτουργίες 2

- Το νήμα A εκτελεί την ακολουθία εντολών A1, A2, A3
- Το νήμα B εκτελεί την ακολουθία εντολών B1, B2, B3
- Όμως η εκτέλεση είναι ταυτόχρονη, επομένως μπορεί να συμβεί και αυτή η σειρά εκτέλεσης:

1:A1, 2:B1, 3:B2, 4:A2, 5:B3, 6:A3

ThreadA :

```
...  
A1  load [counter],r  
  
A2  inc r  
  
A3  store [counter],r  
...
```

ThreadB :

```
...  
  
B1  load [counter],r  
B2  inc r  
  
B3  store [counter],r  
...
```

Παρεμβολή: συμπέρασμα

- Ξέρουμε ότι οι εντολές A1, A2, A3 θα εκτελεστούν ακολουθιακά, αφού ανήκουν στο ίδιο νήμα, A.
- Ξέρουμε ότι οι εντολές B1, B2, B3 θα εκτελεστούν ακολουθιακά, αφού ανήκουν στο ίδιο νήμα, B.
- Αλλά **δεν ξέρουμε** τη σειρά εκτέλεσης των εντολών (A1, A2, A3) ? (B1, B2, B3) συνολικά.
- Ορισμένες σειρές εκτέλεσης είναι `σωστές` και κάποιες άλλες `λανθασμένες`.
- Το φαινόμενο των `λανθασμένων` σειρών εκτέλεσης λέγεται παρεμβολή.

Παράδειγμα παρεμβολής

- Έστω δυο νήματα A και B που εκτελούν ταυτόχρονα τον παρακάτω κώδικα

```
// shared variable  
counter = 0
```

```
main:
```

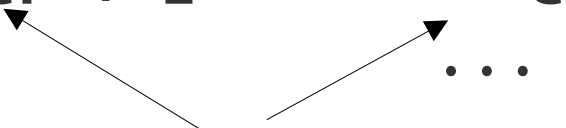
```
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :
```

```
    ...  
    counter = counter + 1  
    ...
```

```
ThreadB :
```

```
    ...  
    counter = counter + 1  
    ...
```



Κρίσιμα τμήματα



Αμοιβαίος αποκλεισμός: εισαγωγή

Βασικοί μηχανισμοί ταυτοχρονισμού

- **Αμοιβαίος Αποκλεισμός**

Πώς διασφαλίζουμε ότι όταν ένα νήμα εκτελεί ένα κρίσιμο τμήμα κάποιο άλλο νήμα δεν εκτελεί αντίστοιχο κρίσιμο τμήμα;

Μας ενδιαφέρει να μην υπάρχει διαπλοκή στην εκτέλεση αντίστοιχων κρίσιμων τμημάτων, όχι η σειρά της εκτέλεσης.

- **Συγχρονισμός σε Συνθήκες***

- Μας ενδιαφέρει όχι μόνο να μην υπάρχει διαπλοκή στην εκτέλεση αλλά και με ποια σειρά θα εκτελέσουν τα νήματα τα κρίσιμα τμήματα.

*θα το δούμε αργότερα

Αμοιβαίος αποκλεισμός (Mutual exclusion)

- Η συνηθέστερη λύση για την αντιμετώπιση των συνθηκών συναγωνισμού.
- Δύο ή περισσότερα κρίσιμα τμήματα διαφορετικών διεργασιών / νημάτων όταν εισέρχονται σε κρίσιμο τμήμα αλληλοαποκλείονται.
- Μόνο μια διεργασία / ένα νήμα κάθε φορά βρίσκεται μέσα σε συγκεκριμένο κρίσιμο τμήμα, και μόνο όταν βγει μπορεί να μπει μια άλλη διεργασία / ένα νήμα.
- Ορίζουμε ένα υποσύνολο των εναλλακτικών ακολουθιακών διαδρομών εκτέλεσης ως νόμιμο.

Αμοιβαίος αποκλεισμός: αποφυγή παρεμβολής 1

```
// shared variable  
counter = 0
```

main:

```
create/start ThreadA  
create/start ThreadB
```

ThreadA :

```
...  
enter_critical_section  
counter = counter + 1  
exit_critical_section  
...
```

ThreadB :

```
...  
enter_critical_section  
counter = counter + 1  
exit_critical_section  
...
```

- Τα `enter_critical_section`, `exit_critical_section` είναι **ατομικές ενέργειες**, δηλαδή δεν διακόπτονται: είτε επιτυχάνουν είτε όχι.

Αμοιβαίος αποκλεισμός: αποφυγή παρεμβολής 2

ThreadA :

...

το νήμα A δεν εισέρχεται

enter_ctitical_section

A1 load [counter],r

A2 inc r

A3 store [counter],r

exit_ctitical_section

...

ThreadB :

...

enter_ctitical_section

B1 load [counter],r

B2 inc r

B3 store [counter],r

exit_ctitical_section

...

το νήμα B δεν εισέρχεται

- Η κατάληψη κρίσιμου τμήματος δεν επηρεάζεται από χρονοδιακοπή, αναστολή, γενικά από δρομολόγηση.

Αμοιβαίος αποκλεισμός: αποφυγή συναγωνισμού δεδομένων

```
//shared variable  
my_data = 0
```

```
main:
```

```
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :
```

```
    ...  
    enter_ctitical_section  
    my_data = 1  
  
    exit_ctitical_section  
    ...
```

```
ThreadB :
```

```
    enter_ctitical_section  
    if my_data == 1...  
        (ή local_var = my_data)  
        (ή my_data = 2)  
    exit_ctitical_section  
    ...
```

Μηχανισμοί αμοιβαίου αποκλεισμού

Δύο βασικοί μηχανισμοί

- Κλειδώματα (Locks) (υψηλού επιπέδου)
- Δυαδικοί Σηματοφόροι (Binary Semaphores) ή Mutexes
- Ορίζονται σαν μοιραζόμενες μεταβλητές
- Οι λειτουργίες τους είναι σύνθετες ατομικές ενέργειες

Κλείδωμα (υψηλού επιπέδου)* 1

- Ένα κλείδωμα είναι ένας ειδικός τύπος δεδομένων `lock_type` που αρχικοποιείται ως μη-δεσμευμένο.
- Ένα νήμα μπορεί να προσπελάσει ένα κλείδωμα μόνο με δύο σύνθετες ατομικές ενέργειες:
- `Lock(s), acquire(s)` : προσπάθεια για δέσμευση, κατάληψη
- `Unlock(s), release(s)`: αποδέσμευση, απελευθέρωση

*Τα κλειδώματα (όπως και οι σηματοφόροι) βασίζονται στα κλειδώματα γλώσσας μηχανής (χαμηλού επιπέδου) που έχουν πιο σύνθετη λειτουργία.

Κλείδωμα (υψηλού επιπέδου) 2

- Lock(M)

↓ μετά την αναστολή νέα ερώτηση
ΑΝ (το M είναι δεσμευμένο)
ΤΟΤΕ αναστολή εκτέλεσης στο M
ΑΛΛΙΩΣ το M δεσμεύεται
είσοδος στο κρίσιμο τμήμα

- Unlock(M)

Εκτελείται από το νήμα που κατέχει το κλείδωμα

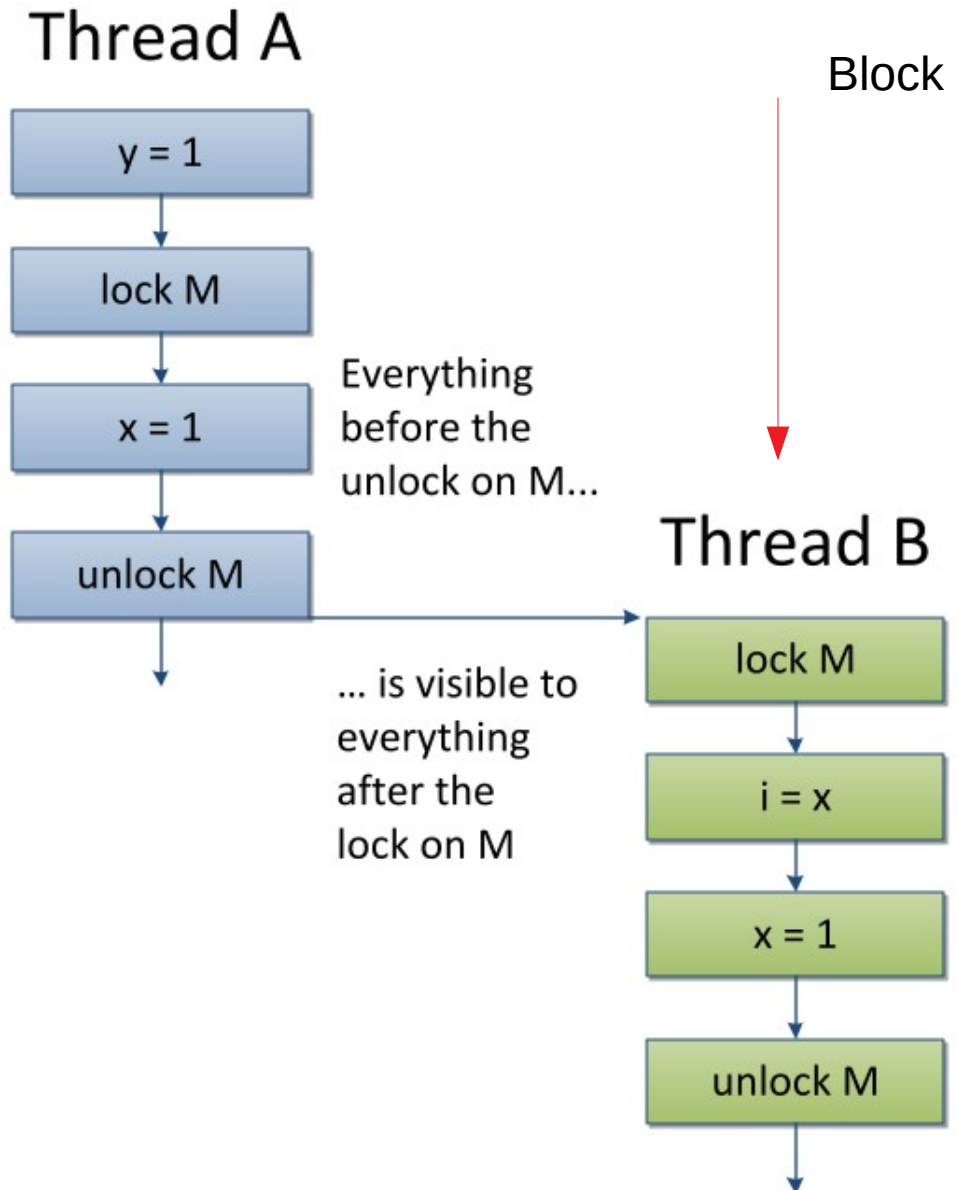
Αποδέσμευση του M

Σε ορισμένες υλοποιήσεις **αν** υπάρχουν νήματα που έχουν ανασταλεί στο M αφυπνίζονται, συνήθως τα νήματα **αφυπνίζονται μόνα τους** μετά από ένα χρονικό διάστημα.

Κλειδώματα (υψηλού επιπέδου) 3

Εκτός από αμοιβαίο αποκλεισμό εξασφαλίζει και:

- Ενημέρωση των μοιραζόμενων μεταβλητών στο Unlock()-Lock()
- Διατήρηση εντός σειράς της εκτέλεσης των εντολών στο Lock()-Unlock() από τον μεταγλωττιστή.



Δυαδικός σηματοφόρος (Binary semaphore)*

Mutex 1

- Ένας σηματοφόρος είναι ένας ειδικός τύπος δεδομένων `sem_type` που λαμβάνει τιμές 0, 1. Αρχικοποίηση $s = 1$.
- Ένα νήμα μπορεί να προσπελάσει ένα mutex μόνο με δύο σύνθετες ατομικές ενέργειες:
- $P(s)$, $acquire(s)$, $down(s)$: προσπάθεια για είσοδο
- $V(s)$, $release(s)$, $up(s)$: έξοδος

*Στη πλήρη του μορφή ο Counting Semaphore (Σηματοφόρος Μετρητής) προτάθηκε από το Dijkstra το 1968, εξ' ου και τα Ολλανδικά αρχικά P, V. Συζητείται σε επόμενη ενότητα.

Δυαδικός σηματοφόρος (Binary semaphore)

Mutex 2

- $P(s)$

AN ($s == 0$)

TOTE αναστολή εκτέλεσης στο s

ΑΛΛΙΩΣ είσοδος στο κρίσιμο τμήμα

↑
μετά την αφύπνιση από την αναστολή

- $V(s)$

AN υπάρχουν νήματα που έχουν ανασταλεί στο s , ένα αφυπνίζεται, ώστε ένα να εισέλθει στο κρίσιμο τμήμα

ΑΛΛΙΩΣ $s = 1$

Δυαδικός σηματοφόρος (Binary semaphore)

Mutex 3

Τα $P(s)$, $V(s)$ έχουν λειτουργία ανάλογη με τα $Lock()$, $Unlock()$, δηλαδή:

- Επιβάλλουν αμοιβαίο αποκλεισμό.
- Επιβάλλουν ενημέρωση των τιμών των μοιραζόμενων μεταβλητών που τροποποιήθηκαν.
- Επιβάλλουν εκτέλεση των εντολών μεταξύ $P(s)$, $V(s)$ εντός σειράς.

Αμοιβαίος αποκλεισμός: υλοποίηση με κλείδωμα

```
// shared variables
```

```
counter = 0
```

```
lock_type l;
```

```
main:
```

```
    create/start ThreadA
```

```
    create/start ThreadB
```

```
ThreadA :
```

```
    ...
```

```
    lock(l)
```

```
    counter = counter + 1
```

```
    unlock(l)
```

```
    ...
```

```
ThreadB :
```

```
    ...
```

```
    lock(l)
```

```
    counter = counter + 1
```

```
    unlock(l)
```

```
    ...
```

Παρόμοια και για το συναγωνισμό δεδομένων.

Αμοιβαίος αποκλεισμός: υλοποίηση με δυαδικό σηματοφόρο

```
// shared variables
```

```
counter = 0
```

```
sem_type s(1);
```

```
main:
```

```
    create/start ThreadA
```

```
    create/start ThreadB
```

```
ThreadA :
```

```
    ...
```

```
    P(s)
```

```
    counter = counter + 1
```

```
    V(s)
```

```
    ...
```

```
ThreadB :
```

```
    ...
```

```
    P(s)
```

```
    counter = counter + 1
```

```
    V(s)
```

```
    ...
```

Παρόμοια και για το συναγωνισμό δεδομένων.

Γενικοί κανόνες αμοιβαίου αποκλεισμού

Κρίσιμα τμήματα

- Κάθε τροποποίηση και ανάγνωση / εγγραφή μοιραζόμενης μεταβλητής πρέπει να ελέγχεται, και να προστατεύεται αν απαιτείται. Μόνο οι απλές αναγνώσεις είναι ασφαλείς.
- Τα κρίσιμα τμήματα δεν πρέπει να είναι πολύ μεγάλα: μειώνουν τον ταυτοχρονισμό. Συγκέντρωση των προσπελάσεων σε συγκεκριμένα τμήματα κώδικα. Όχι αδόμητες κρίσιμες περιοχές.
- Λύνουμε το πρόβλημα στο υψηλότερο δυνατό επίπεδο. Αν υπάρχουν τύποι δεδομένων, δομές δεδομένων ή άλλες βιβλιοθήκες που διασφαλίζουν ασφαλή ταυτόχρονη πρόσβαση τότε τις χρησιμοποιούμε.

Γενικοί κανόνες αμοιβαίου αποκλεισμού

Χρήση lock / mutex

- Το lock / mutex δηλώνονται μοιραζόμενα, ακριβώς όπως και οι μοιραζόμενες μεταβλητές που προστατεύουν.
- Κάθε lock / mutex σχετίζεται με συγκεκριμένη μεταβλητή / δομή δεδομένων / αντικείμενο. Δεν χρησιμοποιείται για άλλες μεταβλητές.
- Όταν έχουμε μεταβλητές που τροποποιούνται πάντα μαζί χρησιμοποιούμε ένα lock / mutex για όλες αυτές τις μεταβλητές.
- Το lock / mutex προστατεύει μεταβλητές όχι τον κώδικα. Δεν κλειδώνουμε τη διεργασία ή το νήμα αλλά τις μεταβλητές που προσπελάζει το νήμα.

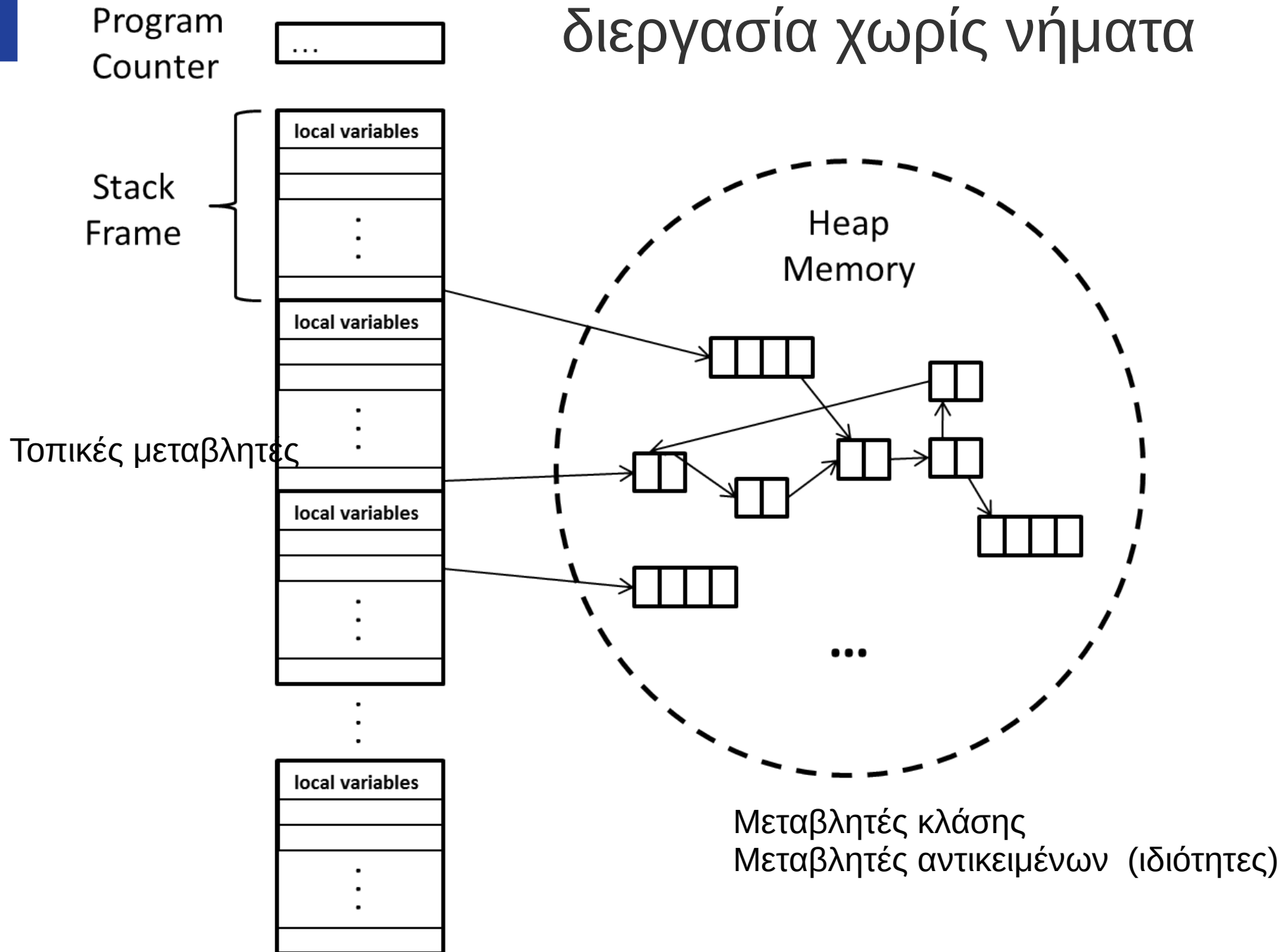
Χρήση lock / mutex χωρίς (εμφανείς) μοιραζόμενες μεταβλητές

- Μερικές φορές ο μοιραζόμενος πόρος δεν έχει τη μορφή μοιραζόμενων μεταβλητών.
- Παράδειγμα: διαπλεκόμενες εκτυπώσεις.
Μοιραζόμενος πόρος είναι η οθόνη (μέσω των κλήσεων συστήματος print).
- Κρίσιμο τμήμα: οι κλήσεις συστήματος εκτύπωσης.
- Το lock / mutex “προστατεύει” τον μοιραζόμενο πόρο μέσω του αμοιβαίου αποκλεισμού του κρίσιμου τμήματος.
- Αναγκαστικά κλειδώνουμε τη διεργασία ή το νήμα.
Προσοχή: το κλείδωμα πρέπει να είναι μοιραζόμενο.

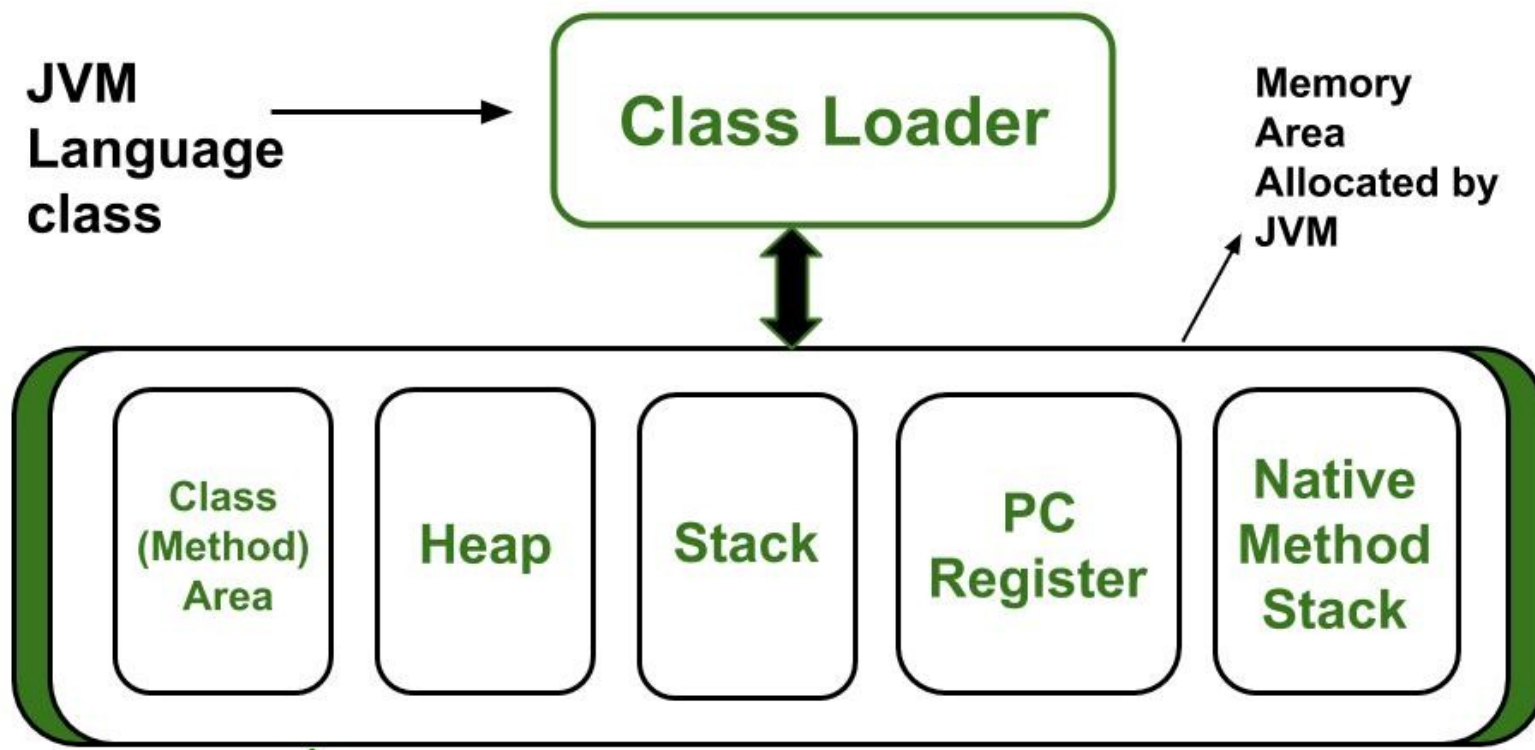


Μοιραζόμενες μεταβλητές στη Java

Χώρος διευθύνσεων Java: διεργασία χωρίς νήματα



Χώρος διευθύνσεων Java 1



Χώρος διευθύνσεων Java 2

- Περιοχή κλάσεων / μεθόδων (Class / Method area): αποθηκεύει τον κώδικα των κλάσεων, αντιστοιχεί με το Text ενός τυπικού χώρου διευθύνσεων.
- Σωρός (Heap): αποθηκεύει μεταβλητές κλάσεων (static), αντικείμενα (μεταβλητές στιγμιοτύπων / αντικειμένων, δομές δεδομένων). Χονδρικά αντιστοιχεί στο συνδυασμό Data και Heap ενός τυπικού χώρου διευθύνσεων.
- Στοίβα κλήσεων (Call Stack): αποθηκεύει τοπικές μεταβλητές μεθόδων και γενικά ότι είναι τοπικό σε μια μέθοδο που εκτελείται (πχ ορίσματα μεθόδων). Κάθε νήμα έχει τη δική του στοίβα κλήσεων.

Χώρος διευθύνσεων Java 3

- Περιοχή μετρητή προγράμματος (PC register area): το `main` (και κάθε νήμα) έχει το δικό του μετρητή προγράμματος
- Περιοχή `native methods`: περιοχή μνήμης για μεθόδους χαμηλού επιπέδου υλοποιημένες σε C.

Συνηθισμένοι τύποι μεταβλητών στη Java 1

- Μεταβλητές τοπικές σε μέθοδο

Γνωστές σε μια μέθοδο – βρίσκονται στο call stack. Έχουν διάρκεια ζωής όσο η μέθοδος.

- Ειδική περίπτωση: μεταβλητές τοπικές στη main()

Γνωστές στη main. Έχουν διάρκεια ζωής όσο η main, δηλαδή συνήθως όσο εκτελείται το πρόγραμμα. Η main είναι static, δηλαδή είναι μοναδική για κάθε πρόγραμμα. Άρα και οι μεταβλητές τις. Οι τοπικές μεταβλητές της main μπορεί να περάσουν ως ορίσματα στα νήματα, που συνήθως δημιουργούνται από τη main.

Συνηθισμένοι τύποι μεταβλητών στη Java 2

- Μεταβλητές αντικειμένου (στιγμιοτύπου)

Οι μεταβλητές που συνήθως δηλώνονται σε μια κλάση και αρχικοποιούνται με τον κατασκευαστή. Κάθε αντικείμενο μπορεί να έχει διαφορετικές τιμές για αυτές τις μεταβλητές (ιδιότητες, attributes).

- Μεταβλητές κλάσης

Οι μεταβλητές αντικειμένου που όμως έχουν την ίδια τιμή για όλα τα αντικείμενα της κλάσης. Δηλώνονται με το προσδιοριστικό `static`.

Το πλησιέστερο στις καθολικές μεταβλητές που διαθέτει η Java.

Συνηθισμένοι τύποι μεταβλητών στη Java 3

- Ορίσματα

Κατά τη κλήση μιας μεθόδου και κατά τη δημιουργία ενός αντικειμένου (κλήση κατασκευαστή) μπορούμε να περάσουμε ορίσματα.

- Ορίσματα τιμής

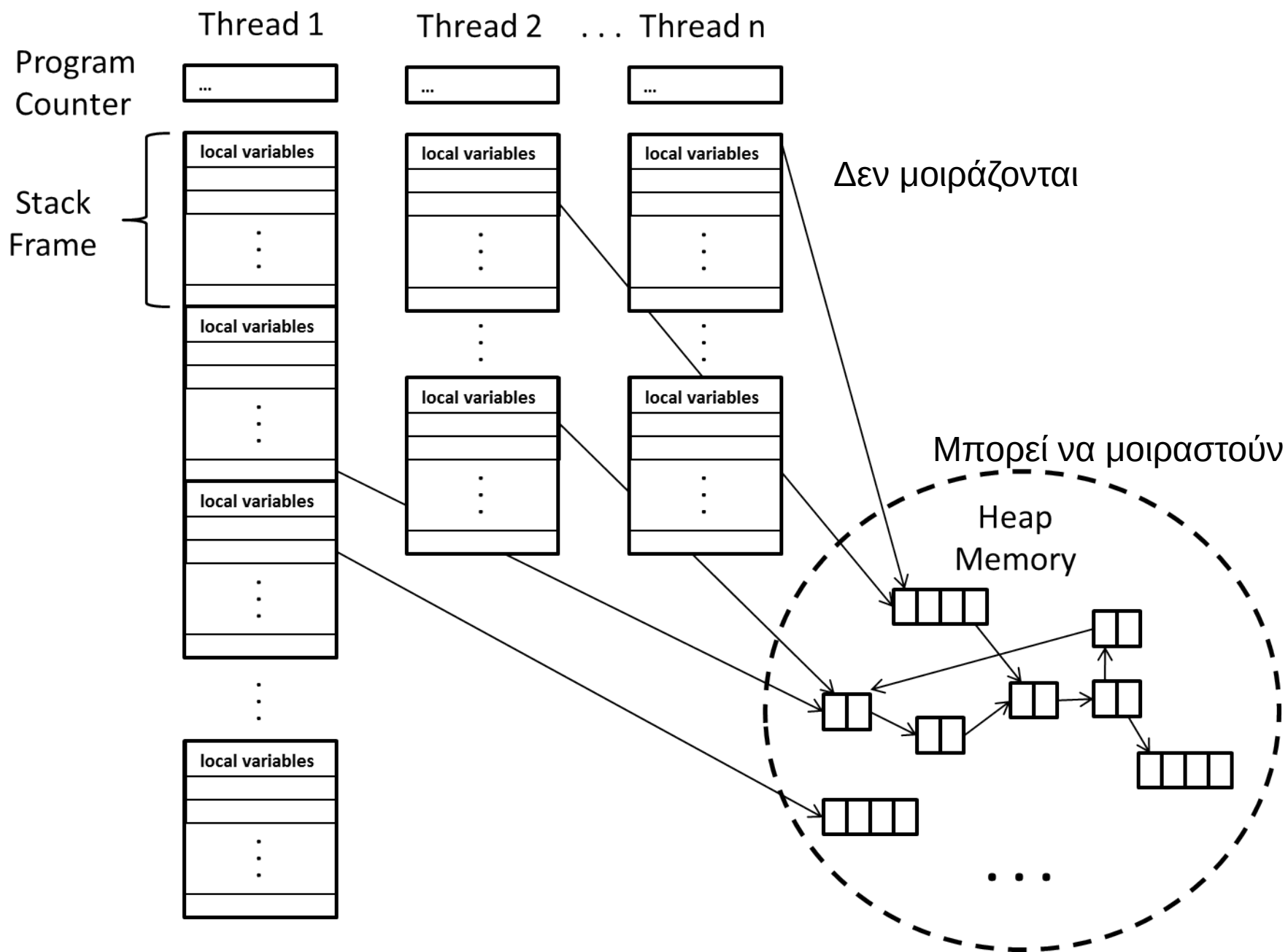
Απλές μεταβλητές (primitive types πχ int, float, char)

- Ορίσματα αναφοράς

Δομές δεδομένων (πχ πίνακες, strings) αλλά και αντικείμενα.

- Τα ορίσματα ενός αντικειμένου συνδέονται (αρχικοποιούν) τις μεταβλητές αντικειμένου μέσω του κατασκευαστή.

Χώρος διευθύνσεων Java: διεργασία με νήματα



Μοιραζόμενες μεταβλητές στη Java 1

Ορίσματα από τη main

- Συνήθως η main δημιουργεί τα νήματα.
- Κατά τη δημιουργία των νημάτων η main μπορεί να τους περάσει ως ορίσματα, απλές μεταβλητές ή δομές δεδομένων ή αντικείμενα που έχουν οριστεί στη main ως τοπικές μεταβλητές (της main).
- Τα νήματα είναι αντικείμενα. Τα ορίσματα που περνά το main στο κάθε νήμα αρχικοποιούν μέσω του κατασκευαστή τις μεταβλητές αντικειμένου (στιγμιοτύπου) του νήματος, οι οποίες είναι πια προσπελάσιμες από τις μεθόδους του νήματος (πχ τη run)

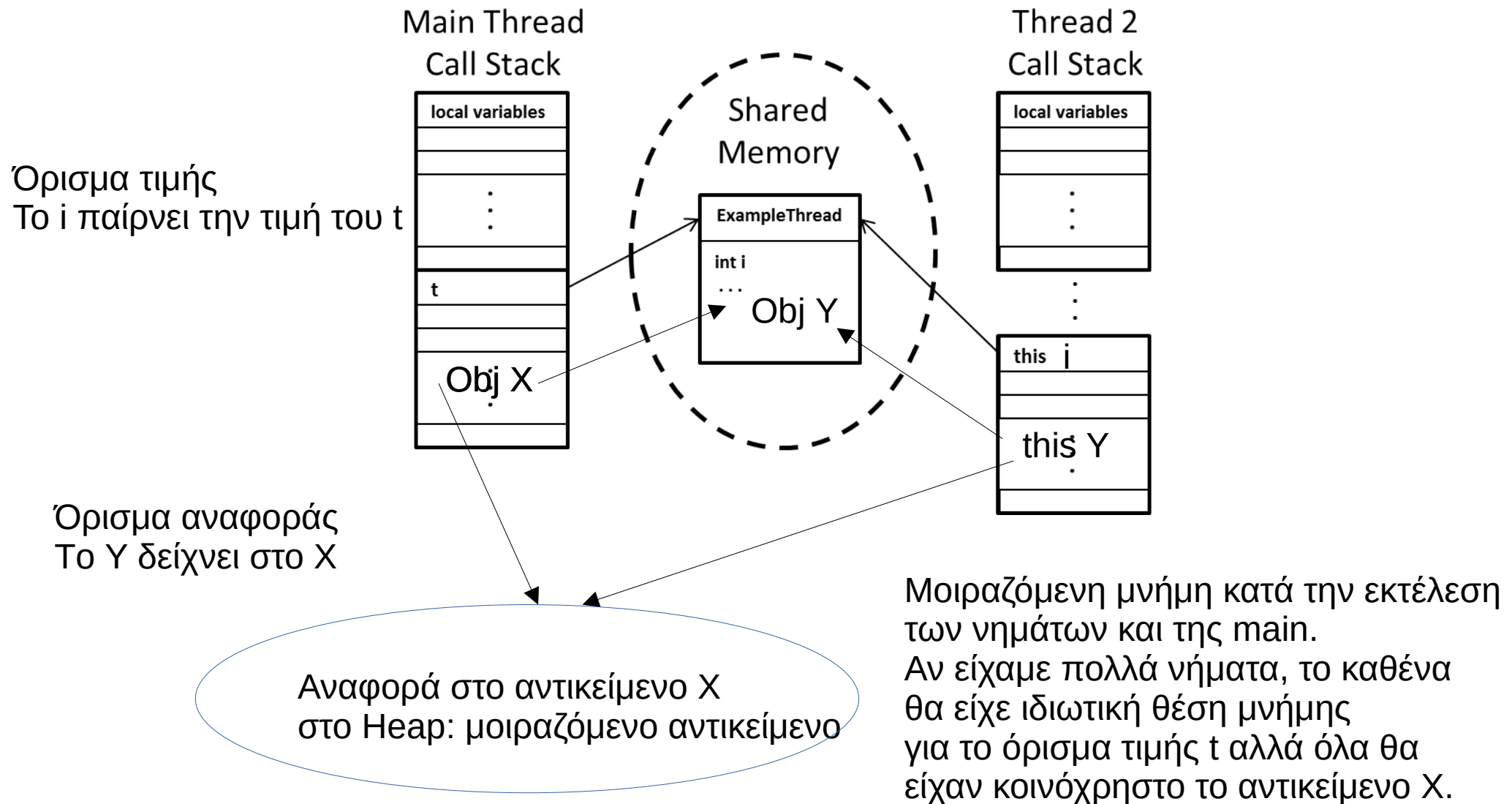
Μοιραζόμενες μεταβλητές στη Java 2

Μηχανισμός μοιραζόμενης μνήμης

- Η `main` διαθέτει μια αναφορά στο δημιουργούμενο νήμα. Επίσης μέσω των ορισμάτων του κατασκευαστή αρχικοποιεί τις μεταβλητές αντικειμένου του νήματος.
- Όμως, το νήμα, σε αντίθεση με τα απλά αντικείμενα, διαθέτει δικό του πλαίσιο εκτέλεσης (program counter, call stack κλπ).
- Επομένως το πέραςμα των ορισμάτων και η αρχικοποίηση των μεταβλητών αντικειμένου υλοποιούν τον μηχανισμό μοιραζόμενης μνήμης της Java: Η `main` περνά τιμές και αναφορές στο νήμα, το οποίο μετά τα χρησιμοποιεί ανεξάρτητα.

Ο μηχανισμός της μοιραζόμενης μνήμης: ορίσματα της main

Η main δημιουργεί νήμα και του περνά ορίσματα μέσω μοιραζόμενης μνήμης κατά τη κατασκευή του νήματος



Μοιραζόμενες μεταβλητές στη Java 3

- Όλες οι δομές δεδομένων και τα αντικείμενα αποτελούν ορίσματα αναφοράς εξ' ορισμού.
- Οι απλές μεταβλητές είναι τιμές, επομένως δεν περνούν ως ορίσματα αναφοράς αλλά τιμής. Αν θέλουμε να τις διαμοιράσουμε ή τις εντάσσουμε σε αντικείμενα ή τις καθιστούμε καθολικές μεταβλητές.

Μοιραζόμενες μεταβλητές στη Java 4

Επιστροφές τιμών

- Όταν δεν υπάρχουν ορίσματα αναφοράς πρέπει να γίνει επιστροφή τιμών από τον καλούμενο στον καλούντα.
- Στη Java, στα απλά νήματα, δεν υπάρχει επιστροφή τιμών. Αυτή μπορεί να γίνει μέσω μεταβλητών αντικειμένου που είναι γνωστές στον καλούντα (συνήθως το main).
- Καλό είναι οι επιστροφές να γίνονται μέσω get().
- Θα δούμε αργότερα ότι υπάρχουν κλάσεις που επιστρέφουν αντικείμενα.

Μοιραζόμενες μεταβλητές στη Java 5

Ταυτόχρονη πρόσβαση μέσω ορισμάτων αναφοράς

- Απαιτείται ιδιαίτερη προσοχή στη χρήση ή και προστασία.
- Το ένα νήμα μπορεί να επηρεάσει τη δουλειά του άλλου (συμπεριλαμβανομένου και του `main`)
- Μια λύση είναι κάθε νήμα να γράφει σε ξεχωριστή περιοχή της δομής δεδομένων. Η μοιραζόμενη ανάγνωση δεν μας προβληματίζει, αλλά όταν υπάρχει ανάγνωση-εγγραφή ή εγγραφή-εγγραφή έχουμε πρόβλημα.

Μοιραζόμενες μεταβλητές στη Java 6

Καθολικές μεταβλητές

- Static μεταβλητές (στη κύρια κλάση, του main)

Οι μεταβλητές της κύριας κλάσης του προγράμματος είναι static, δηλαδή μοναδικές (όπως και η μέθοδος main).

Είναι ορατές (και μοιραζόμενες) στο main και σε πιθανές εσωτερικές κλάσεις (και στα αντικείμενά τους).

Είναι προσπελάσιμες και από εξωτερικές κλάσεις (και τα αντικείμενά τους) με πρόθεμα το όνομα της κύριας κλάσης.

Το πλησιέστερο στις καθολικές μεταβλητές στην Java. Συνηθισμένη λύση διαμοιρασμού για primitive μεταβλητές.



Παραδείγματα

- Code/Shared

Ειδικά θέματα διαμοιρασμού μεταβλητών στη Java 1

- Γενικά κάθε γλώσσα έχει τις ιδιαιτερότητές της, τόσο στον τρόπο δήλωσης όσο και στον χειρισμό των μεταβλητών. Χρειάζεται προσοχή στις ειδικές περιπτώσεις.
- Ειδικά η Java έχει πολλές ιδιαιτερότητες λόγω της αντικειμενοστρέφειας, της κληρονομικότητας (extends), της ενθυλάκωσης (public/private), των εσωτερικών (inner) κλάσεων, του static και άλλων περιπτώσεων που τροποποιούν την συμπεριφορά των μεταβλητών.
- Θέλει ιδιαίτερη προσοχή και κατανόηση του κώδικα, και γράφουμε απλό κώδικα!

Ειδικά θέματα διαμοιρασμού μεταβλητών στη Java 2

- Static μεταβλητές (άλλων κλάσεων, πχ νημάτων)
Μοιραζόμενες σε όλα τα στιγμιότυπα (αντικείμενα) της κλάσης.

Αν η κλάση του νήματος έχει static μεταβλητές τότε αυτές είναι μοιραζόμενες στα νήματα, σε εσωτερικές κλάσεις (και στα αντικείμενά τους). Εύκολη λύση αλλά θέλει προσοχή: τεκμηρίωση στον κώδικα.

Προσοχή σε 'αυθαιρεσίες' πχ τροποποίηση static μεταβλητής νημάτων: αυτό θα επηρεάσει όλα τα νήματα.

Ειδικά θέματα διαμοιρασμού μεταβλητών στη Java 3

- Μεταβλητές αντικειμένου (άλλων κλάσεων, και ειδικότερα νημάτων)

Ξεχωριστές για κάθε στιγμιότυπο (αντικείμενο) της κλάσης.

Οι μεταβλητές στιγμιοτύπου (αντικειμένου) των νημάτων μπορούν να προσπελαστούν από άλλα αντικείμενα.

Προσοχή σε ‘αυθαιρεσίες’ πχ τροποποίηση μεταβλητής αντικειμένου ενός νήματος από άλλο νήμα ή το main. Προστασία με προσδιοριστικό `private`.

Ειδικά θέματα διαμοιρασμού μεταβλητών στη Java 4

- Static μέθοδοι

Μοιραζόμενες σε όλα τα στιγμιότυπα (αντικείμενα) της κλάσης: κώδικας και μεταβλητές.

Αν η κλάση του νήματος έχει static μεθόδους τότε αυτές είναι μοιραζόμενες στα νήματα, όπως και οι μεταβλητές τους.

- Static κλάσεις

Τα αντικείμενα static κλάσεων συμπεριφέρονται όπως και αυτά των non-static κλάσεων.



Αμοιβαίος αποκλεισμός στη Java

Βασικοί μηχανισμοί ταυτοχρονισμού στη Java 1

- Ενσωματωμένοι στη κύρια γλώσσα

synchronized: Αμοιβαίος αποκλεισμός

wait-notify: Συγχρονισμός σε συνθήκη

Οι δύο μηχανισμοί συνθέτουν τον επόπτη (monitor).

- `java.util.concurrent`

Βιβλιοθήκη με πολλούς επιπλέον μηχανισμούς, δομές δεδομένων, ατομικές ενέργειες κλπ. Δείτε

https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html#package_description

- Δοκιμάστε το `javaConcurrentAnimated.jar`

<https://www.jconcurrent.com/>

Κλασσικός αμοιβαίος αποκλεισμός στη Java

- Τα κλειδώματα και οι σηματοφόροι στη Java δεν είναι ενσωματωμένοι στην κύρια γλώσσα αλλά βρίσκονται στο πακέτο `java.util.concurrent`
- Οι σχεδιαστές της Java προσπάθησαν με το `synchronized` να δώσουν έναν υψηλότερου επιπέδου αντικειμενοστρεφή μηχανισμό αμοιβαίου αποκλεισμού.
- Όμως η σωστή χρήση του `synchronized` δεν είναι προφανής, ειδικά όταν αναφερόμαστε σε primitive τύπους.
- Γι' αυτό θα ξεκινήσουμε με τους κλασσικούς μηχανισμούς: κλείδωμα και δυαδικό σηματοφόρο.

Κλασσικό κλείδωμα στη Java 1

- Στη Java το Lock interface υλοποιείται από τα ReentrantLock, ReadWriteLock, ReentrantReadWriteLock στο πακέτο `java.util.concurrent`.
- Εδώ χρησιμοποιούμε μόνο το ReentrantLock.
- Reentrant: το ίδιο νήμα μπορεί να ξαναπροσπαθήσει να καταλάβει κλείδωμα που ήδη κατέχει χωρίς να προκληθεί σφάλμα
- Βασικές μέθοδοι: `lock()`, `unlock()`
- Παρέχονται επιπλέον μέθοδοι δίνουν ευελιξία στη χρήση, δεν καλύπτονται εδώ.

Κλασσικό κλείδωμα στη Java 2

```
import java.util.concurrent.locks.Lock;  
import java.util.concurrent.locks.ReentrantLock;
```

```
Lock lock = new ReentrantLock();
```

```
// προσπάθεια για κατάληψη κλειδώματος  
lock.lock();  
try {
```

```
    ... //critical section
```

```
    // απελευθέρωση κλειδώματος  
} finally {  
    lock.unlock();  
}
```

Το finally {} διασφαλίζει απελευθέρωση του κλειδώματος αν συμβεί εξαίρεση, διακοπή, αν υπάρχει break, continue, return.

Κλασσικό κλείδωμα στη Java 3

Εναλλακτική διατύπωση (δεν συνιστάται).

```
import java.util.concurrent.locks.Lock;  
import java.util.concurrent.locks.ReentrantLock;
```

```
Lock lock = new ReentrantLock();
```

```
// προσπάθεια για κατάληψη κλειδώματος  
lock.lock();
```

```
... //critical section
```

```
// απελευθέρωση κλειδώματος  
lock.unlock();
```

Σηματοφόροι στη Java 1

- Στη Java οι σηματοφόροι Semaphore, βρίσκονται στο πακέτο `java.util.concurrent`.
- Είναι αντικείμενα που ορίζονται ως
`Semaphore sem = new Semaphore(permits, true);`
- `permits`: μη- αρνητικός αριθμός, αρχική τιμή.
- `true`: δίκαιος σηματοφόρος (προαιρετικό όρισμα).
- Βασικές μέθοδοι: `acquire()`, `release()`
- Παρέχονται πολλές επιπλέον μέθοδοι που δίνουν ευελιξία στη χρήση, δεν καλύπτονται εδώ.
- Δεν υπάρχει ειδική κλάση δυαδικού σηματοφόρου.

Σηματοφόροι στη Java 2

```
import java.util.concurrent.Semaphore;

Semaphore sem = new Semaphore(capacity, true)

...
try {
    sem.acquire();
    ... //critical section

} finally
    sem.release();
} catch (InterruptedException e) { }
...
```

Το finally {} διασφαλίζει απελευθέρωση του σηματοφόρου αν συμβεί εξαίρεση, διακοπή, αν υπάρχει break, continue, return.

Σηματοφόροι στη Java 3

Εναλλακτική διατύπωση (δεν συνιστάται).

```
import java.util.concurrent.Semaphore;

Semaphore sem = new Semaphore(capacity, true)

...
try {
    sem.acquire();
    ... //critical section
} catch (InterruptedException e) { }

sem.release();
...
```

Παραδείγματα κλασσικού αμοιβαίου αποκλεισμού στη Java

- Code/SharedCounter
- Code/SharedCounterMutex

Περιέχονται διάφορες εναλλακτικές μοιραζόμενης μνήμης και αμοιβαίου αποκλεισμού που συζητήθηκαν.